Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science Year 11 2026

Programming 9 — First Random Number

Every program you have written so far does the same thing every time you give it the same input. A dice game, a quiz that picks its own questions, or a game where the computer hides a number needs something new: a number the program chooses for itself, which nobody can predict.

1 Your first random number

> VOCAB **Random number generation** (OCR 2.2.3) — a program choosing a number for itself from a range you give it, so the result cannot be predicted in advance

The rule. Write `import random` once, on the first line of the program. Then `random.randint(low, high)` gives a whole number from `low` to `high`, **and both ends are included**. Store it in a variable and use it like any other value.

```
import random

roll = random.randint(1, 6)
print("You rolled", roll)
```

Line 1 brings in Python's `random` module, a set of ready-made sub programs. `randint` is short for *random integer*. `randint(1, 6)` can give 1, 2, 3, 4, 5 or 6: it includes the 6, where `range(1, 6)` stops at 5.

Now try • Type it in and press Run five times. Then change it into a 20-sided dice.

Check • The five runs give numbers from 1 to 6, and not all the same. The 20-sided version gives numbers from 1 to 20.

Now try • A bank card PIN is a number from **1001** to **9999**. Write a program that makes a random PIN and prints it.

Check • Run it several times: every PIN printed has exactly four digits.

main.py

```
1
2
3
4
```

2 Using the random number

The rule. A random number is just a number, so it can go anywhere a number goes: in an `if`, in a calculation, or as an index into a list. To choose between outcomes, give each outcome its own number.

Toss a coin. Two outcomes, so two numbers: 1 stands for heads, 2 for tails.

```
import random

flip = random.randint(1, 2)
if flip == 1:
    print("Heads")
else:
    print("Tails")
```

Pick from a list. `random.choice()` picks one item from a list for you.

```
import random

moves = ["rock", "paper", "scissors"]
computer = random.choice(moves)
print("The computer plays", computer)
```

The exam's ERL has only `random(…,…)`, with no `choice`. So in an exam answer, pick a random **index** instead: `moves[random.randint(0, 2)]` in Python, `moves[random(0,2)]` in ERL. Indexes start at 0, so three items are 0, 1 and 2.

Now try • Make a random **multiple of 5** from 5 to 100, such as 35 or 80. Make a random number from 1 to 20 first, then multiply it by 5.

Check • Run it several times: every answer ends in 0 or 5, the smallest possible is 5 and the largest is 100.

main.py

```
1
2
3
4
```

3 The exam writes it in ERL

OCR's Exam Reference Language uses one word for random numbers, and needs no import line.

Python (what you type)	ERL (what the exam shows)	Gives
`import random`	not written	the random module, ready to use
`random.randint(1, 6)`	`random(1,6)`	a whole number from 1 to 6, both included
`random.uniform(-1.0, 10.0)`	`random(-1.0,10.0)`	a real number from −1.0 to 10.0

ERL tells a whole number from a real number by what you put in the brackets: `random(1,6)` gives an integer, `random(-1.0,10.0)` gives a real.

Now try • This ERL program rolls a dice three times. Write it in Python, then run it.

```
total = 0
for i = 1 to 3
    roll = random(1,6)
    print(roll)
    total = total + roll
next i
print("Total: " + str(total))
```

Check • It prints three numbers, each from 1 to 6, then a total equal to their sum. The total is always between 3 and 18.

main.py

```
1
2
3
4
5
6
7
8
```

4 Guess the Number, counting the guesses

This is your holiday program with one new variable, `guesses`, which goes up by 1 every time round the loop.

```
1  import random
2
3  secret = random.randint(1, 100)
4  guesses = 0
5  guess = 0
6  while guess != secret:
7      guess = int(input("Guess a number from 1 to 100: "))
8      guesses = guesses + 1
9      if guess > secret:
10         print("Too high")
11     elif guess < secret:
12         print("Too low")
13 print("Correct! You took", guesses, "guesses")
```

Line 3 runs once, before the loop, so the secret number stays the same for the whole game. Line 8 is a **counter**: it starts at 0 and adds 1 each pass. When the loop ends, it holds the number of guesses.

Now try • Play three games and record how many guesses each one took.

Check • The secret number changes every game, and the last line reports the same number of guesses that you counted.

Game 1	Game 2	Game 3	My method for choosing each guess

Now try • **The challenge.** Find a method of guessing that **always** wins in **7 guesses or fewer**, whatever the secret number is. Write your method in the table above.

Check • Play two more games with your method: both finish in 7 or fewer. Bring the method to P2: it is the algorithm you meet next.

Stretch A random password

Write a program that makes a password of **6 random capital letters**, such as `QHBWZE`. From Programming 5: `chr(65)` is `"A"` and `chr(90)` is `"Z"`, so `chr(random.randint(65, 90))` is one random capital letter. Use a loop and join each letter on to the end of the password.

Check • Every run prints six capital letters, and a different password each time.

main.py

```
1
2
3
4
5
6
7
```

The exam answers, one line each

Asked for	Write
A random number from 1 to 6	`roll = random(1,6)` in ERL, or `roll = random.randint(1, 6)` in Python.
Are 1 and 6 both possible?	Yes: both ends of the range are included.
A random item from a list	A random index: `moves[random(0,2)]` for a list of three.
What random numbers are for	Making outcomes a program cannot predict: dice, cards, quiz questions, computer players, simulations.